Core canonical identity as one object: `canon.hash()`

Created-at: 2026-09-29 18:49 +02:00 ETA: about 2.5 h (Canon and its identity 0.5 h; 94 Core call sites 0.5 h; tests and gates 0.5 h; Nagare repin, migration and A/B 1 h)

Request. User, 2026-09-29: fetch the newest version of core. The hash should be computed"; create new branch on feature. We need cannon.hash() or more structural and object oriented". Core and its remote were fetched at 18:47; `origin/main` is `9a236ac`, Nagare's pin `fb94550` plus two documentation commits. Branch `feat/canonical-identity` from it, upstream unset.

Evidence.

- `ModuleStore::compile` computes the structural V1 hash (`canon_hash`) and the deprecated declaration Merkle table; it computes no semantic identity. Its own comment names `semantic_model_digest_v2` as the sound identity".
- V1 is blind to values: with Core's CLI, `vertex_index 1→7` and `slot "s"→"t"` keep the hash; one arc sign changes it.
- Identity is a flat family of free functions over `(ir, interner)`: `canonical_program_hash`, `program_structure_hash_v1`, `semantic_model_digest_v2`, `semantic_declaration_hashes`; each derives the canonical paths again (`CanonicalPaths`, `build_path_index`).
- Nagare computed the source digest itself until `faa22b8`; the user placed it in Core.

Design.

- `Canon<'ir>`: the canonical view of one IR – the IR, its interner, its path index derived once. Methods `hash()` (semantic V2 digest), `structure(cfg)` (V1), `declaration_hashes()`, `paths()`, `validate(idx)` (the lowering-boundary check). It replaces the four free functions and the crate-private `CanonicalPaths`.
- `CanonicalIdentity`: the value a compile keeps – `hash()` and `structure()` – computed once in `ModuleStore::compile` through one `Canon`. `CompiledProgram::canon` replaces `canon_hash`: `program.canon.hash()`.
- Unchanged: every hash byte (known-answer vectors), the CBOR wire field `canon_hash` (data, written from `Canon::structure`), the deprecated Merkle pass (classified ambiguous by Core, untouched), `build_path_index`.

CORE.YAML. `hymeko_core` is `lockdown: full`. This change needs `APPROVED-CORE-EDIT: canonical-identity`; its entry is added to `approved_edits` in the same change. Scope: `hymeko_core/src/ir/{canonical_hash,semantic_digest}.rs`, `module_store/module_store.rs`, `writers/cbor_writer.rs` and its `validate.rs`, `util/pretty_print.rs`, `ir/{coherence,time}.rs`, the tests, benches and example that call the replaced functions (94 sites). Constraints: no hash value changes; no wire change; no dependency.

Consumers. Nagare migrates in this work after a local-path build (no push): the source identity is `program.canon`, the model digest `Canon::new(ir, it).hash()`. Listed, not edited, migrate at their next repin: `graph/hymeko_graph/tests/ir_ops_bench.rs`, `transformation/hymeko_emitter/tests/fault_mutants.rs`, `network/src/optimize.rs`, `application/hymeko_daemon/src/worker.rs`.

Tests. Existing known-answer and coverage suites (`frozen_hash.rs`, `semantic_digest.rs`, `identity.rs`, `hash_coverage.rs`) run unchanged through the methods: bit identity. New: a compiled program's `canon` equals `Canon` of its IR; `hash()` separates the two value changes `structure()` cannot. Core gate, then Nagare's `scripts/check.sh` and its eight regression records bit for bit.

Performance budget. Core's compile gains one semantic digest and loses one path derivation. Nagare at CogInfoCom Alpha (seed 0, 20 passes, runner settings): compile at most 13.0 s (the `c1a155e` level, which computed this digest in Nagare), peak RSS 2243 MiB.

Rollback. The branch is unpushed; Nagare keeps its pin until the repin commit.

Risks. Every Core compile now pays for the semantic digest, consumers that never read it included. The four listed consumers break at their next repin until migrated. A method whose bytes differ from the free function it replaces fails the known-answer suites.

